

Ethical AI Storyteller: Integrating Safety, Bias Mitigation, and Explainability in Neural Story Generation

1st Eron Salihu
Computer and Software Engineering
University of Prishtina
Prishtina, Kosovo
eron.salihu@student.uni-pr.edu

2nd Miran Mustafa
Computer and Software Engineering
University of Prishtina
Prishtina, Kosovo
miran.mustafa@student.uni-pr.edu

3rd Urim Hoxha
Computer and Software Engineering
University of Prishtina
Prishtina, Kosovo
urim.hoxha@student.uni-pr.edu

Abstract—Large language models can generate fluent narratives but risk producing unsafe or biased content and provide little transparency about their decisions. This work presents the Ethical AI Storyteller, a system that combines pretrained GPT-2 models with domain-specific storytelling data, a multi-layer ethical filter (Detoxify plus rule-based heuristics), pattern-based bias detection, and lightweight explainability tools (attention and token-importance visualizations). We fine-tune on creative-writing datasets and provide a real-time Gradio interface that surfaces content ratings, bias warnings, and interpretability views. Experiments on a curated storytelling set show improved safety (safe content rate up to 97%) with minimal degradation in perplexity, demonstrating that ethical constraints and usability features can be integrated without sacrificing narrative quality.

Index Terms—Story Generation, Ethical AI, Toxicity Filtering, Bias Detection, Explainable NLP, GPT-2, Content Safety

I. INTRODUCTION

Generative models enable rapid creation of narratives for education, entertainment, and prototyping. However, unfiltered outputs can contain toxicity, harmful stereotypes, or opaque reasoning [4]. We address three deployment challenges: (1) ensuring content safety, (2) detecting and mitigating social biases, and (3) providing transparency for users and instructors. Our contributions are:

- A modular storyteller that combines GPT-2 generation with layered ethical filtering and bias detection.
- Built-in explainability: attention heatmaps, token-importance bars, and next-token diagnostics exposed in the UI.
- A reproducible fine-tuning and evaluation pipeline with lightweight perplexity, safety, and bias metrics.
- A Gradio interface for real-time use, including safety ratings, bias alerts, and explainability plots.

A. Problem statement

Given an input prompt x , a neural generator produces a continuation $y \sim p_\theta(\cdot | x)$. In open-ended generation, harmful content can appear even for innocuous prompts. Our goal is to generate stories that are (i) engaging and coherent, while (ii) reducing unsafe or biased outputs, and (iii) providing interpretable diagnostics to the user.

B. Design goals

The project is scoped for a course setting. Therefore, we prioritize: (a) modularity and testability, (b) offline reproducibility for grading, and (c) transparent user feedback rather than purely automatic content suppression.

II. RELATED WORK

A. Language Model-Based Story Generation

Transformer-based language models have emerged as a leading paradigm for text generation, with GPT-2 [1] and its subsequent variants demonstrating strong performance across a range of narrative and open-ended language tasks. Their effectiveness is primarily attributed to the self-attention mechanism [8], which enables the modeling of long-range contextual dependencies that are difficult to capture using earlier neural architectures. More advanced GPT-3-style models [2] further extend these capabilities through few-shot learning; however, prior studies have shown that this increased flexibility may also intensify the propagation of biases inherent in large-scale training corpora [5].

B. Content Safety and Toxicity Detection

To mitigate the generation of harmful content, toxicity detection frameworks such as Detoxify [3] have been widely adopted, providing robust classification of abusive language, including hate speech, threats, and identity-based attacks. In addition, benchmark datasets such as RealToxicityPrompts [11] have been introduced to systematically evaluate toxic degeneration in neural language models. Motivated by these findings, the proposed system integrates machine-learning-based detection methods with complementary rule-based filtering strategies, with the objective of improving coverage while maintaining precise control over recurrent toxicity patterns.

C. Bias Detection in NLP

Research in bias detection has focused on identifying gender-, racial-, and profession-related stereotypes in generated text [5]. Seminal work on debiasing word embeddings

TABLE I
COMPARISON WITH RELATED STORY GENERATION SYSTEMS.

System	Safety	Bias	Explain	UI
GPT-2 (base)	×	×	×	×
GPT-3 API	Partial	×	×	✓
Perspective API	✓	×	×	×
Story generation libs	×	×	×	Varies
Ours	✓	✓	✓	✓

[13] highlighted how models encode societal biases. CrowS-Pairs [12] provides evaluation datasets for measuring social biases. Our implementation uses pattern-based detection combined with actionable recommendation systems.

D. Explainable AI for Language Models

Attention visualization [6] and token importance analysis [7] have emerged as key techniques for understanding transformer model behavior. Analysis of attention patterns in BERT [14] demonstrated that attention can reveal linguistic structure. Our system implements multiple explainability methods including attention heatmaps, leave-one-out importance analysis, and generation tracing. We use the Hugging Face Transformers stack [9] to ensure reproducibility.

E. Comparison with Existing Systems

Table I compares our approach with related story generation systems. Unlike purely generative approaches, we integrate safety, bias detection, and explainability in a unified framework designed for educational use.

III. PROBLEM FORMULATION

A. Generation

We use causal language modeling for next-token prediction. During inference, we sample tokens with nucleus sampling and temperature scaling. To reduce degeneration, we apply repetition control via no-repeat n -grams and a repetition penalty.

B. Safety and bias objectives

We treat safety as a post-generation constraint. Given a generated candidate y , we compute:

$$\text{rating}(y) = f_{\text{safety}}(y; \tau) \in \left\{ \begin{array}{l} \text{Safe} \\ \text{Mild} \\ \text{Moderate} \\ \text{Restricted} \\ \text{Blocked} \end{array} \right\}$$

and a bias indicator $b(y) = \mathbb{I}[f_{\text{bias}}(y) = \text{biased}]$. The system can optionally re-generate if $\text{rating}(y)$ is unacceptable.

IV. SYSTEM OVERVIEW

Figure 1 illustrates the end-to-end workflow. The operational data flow proceeds as follows:

- 1) The user provides a textual prompt and genre selection.
- 2) The Story Generator produces an initial narrative draft.

Pipeline: Prompt → GPT-2 Generation → Safety Filter (Detoxify/Rules) → Bias Checks → Explainability (Attention/Importance) → UI Output

Fig. 1. Ethical AI Storyteller pipeline with safety, bias, and explainability layers.

TABLE II
DATASETS USED IN THE PROJECT. “LOCAL” IS BUNDLED WITH THE REPOSITORY FOR OFFLINE GRADING.

Dataset	Type	Samples	Notes
WritingPrompts	HF	500–1000	Creative long-form
TinyStories	HF	500–1000	Simple narratives
FairyTales	HF	200–500	Classic story structure
Local (JSON)	Repo	50	Offline, multi-genre

- 3) The Ethical Filter evaluates the generated content for safety and policy compliance.
- 4) The Bias Detector analyzes the narrative for potential stereotypes.
- 5) If unsafe content is detected, the system regenerates with adjusted parameters (max 3 retries).
- 6) The Explainability Module provides insights into the generation and filtering decisions.
- 7) The final story is returned along with a comprehensive ethical analysis.

V. DATASETS AND PREPROCESSING

We leverage three sources: (1) **WritingPrompts** (creative Reddit prompts), (2) **TinyStories** (simple narratives), and (3) a curated **local set** of 50 prompt–completion pairs across multiple genres. The local dataset enables fully offline reproducibility.

A. Cleaning and formatting

We normalize Unicode, remove broken encodings, and strip excessive whitespace. Each example is formatted as `prompt + story`. For lightweight dataset inspection and statistics we rely on standard Python tooling (e.g., pandas) [10]. For training and evaluation, long sequences are truncated to a fixed maximum length.

B. Dataset statistics

Table II summarizes the datasets used in experiments.

VI. SYSTEM ARCHITECTURE

Components communicate through the `EthicalStoryTeller` orchestrator. The design is modular: each capability (generation, filtering, bias checks, explanations) is independently testable and can be enabled or disabled depending on available resources.

A. Module breakdown

Table III lists the key modules and their responsibilities.

TABLE III
CORE MODULES AND RESPONSIBILITIES.

Module	Responsibility
story_generator	Load LM, generate continuations, configure sampling
dataset	Load HuggingFace and custom JSON datasets
finetune	Fine-tuning pipeline with checkpointing
ethical_filter	Safety scoring, bias detection, content rating
explainability	Attention visualization, token-importance
evaluate	Perplexity, safety, and bias metrics
storyteller	Main orchestrator integrating all components
app.py	Gradio user interface

VII. METHODOLOGY

A. Story Generation

The system utilizes GPT-2 as the base language model, supporting multiple variants:

- GPT-2 (124M parameters) – default for efficiency
- GPT-2 Medium (355M parameters)
- GPT-2 Large (774M parameters)
- DistilGPT-2 (82M parameters) – fastest inference

We employ nucleus sampling ($p = 0.95$), top- k (50), temperature (0.8–1.0), and 3-gram repetition blocking. Genre prompts steer tone and structure across Fantasy, Science Fiction, Mystery, Romance, Horror, Adventure, Fairy Tale, and Historical categories.

B. Fine-tuning

Fine-tuning follows standard causal language modeling with AdamW. We train for 2–3 epochs with small batch sizes to fit typical student hardware. Early checkpoints are saved for recovery, and a validation split is used to select the final model.

C. Hyperparameters

Table IV lists the default hyperparameters used in our pipeline. The same configuration is exposed through the CLI and the UI to support controlled experiments.

D. Ethical Filtering

Layer 1 employs Detoxify scores with conservative thresholds (toxicity 0.5, severe toxicity 0.3). Layer 2 applies blocked-pattern regexes for violence, discrimination, and self-harm. Layer 3 encourages prosocial cues (friendship, courage, growth) to promote positive framing. Content is rated in five levels: Safe, Mild, Moderate, Restricted, Blocked.

E. Safety rating function

We combine signals from (i) blocked-pattern matches, (ii) sensitive-topic matches, (iii) Detoxify scores (when enabled), and (iv) presence of prosocial cues. Blocked patterns force a *Blocked* label. Sensitive topics increase severity, while positive cues can increase confidence for *Safe/Mild* ratings. This heuristic design trades recall (catch more potentially harmful content) for simplicity and transparency in a coursework context.

TABLE IV
KEY HYPERPARAMETERS USED IN TRAINING AND INFERENCE.

Component	Setting
Base model	GPT-2
Train epochs	2–3
Batch size	4
Optimizer	AdamW
Learning rate	5×10^{-5}
Max length (train)	256
Max length (eval)	512
Warmup steps	100
Grad clip	1.0
Max generated tokens	200
Temperature	0.8
Top- p	0.95
Top- k	50
No-repeat n -gram	3
Repetition penalty	1.2
Toxicity threshold	0.5
Severe toxicity threshold	0.3

Algorithm 1 Multi-layer ethical filtering

Require: Candidate text y

- 1: $F \leftarrow \{\}$ {flag list}
- 2: **if** matchesBlockedRegex(y) **then**
- 3: **return** Blocked
- 4: **end if**
- 5: $S \leftarrow \text{findSensitiveTopics}(y)$
- 6: **if** $|S| > 0$ **then**
- 7: addFlags(F, S)
- 8: **end if**
- 9: **if** DetoxifyEnabled **then**
- 10: $T \leftarrow \text{detoxifyScores}(y)$
- 11: addHighScoresToFlags(F, T)
- 12: **end if**
- 13: $p \leftarrow \text{countProsocialCues}(y)$
- 14: **return** ratingFrom(F, T, S, p)

F. Filtering algorithm

Algorithm 1 summarizes the multi-layer decision process.

G. Bias Detection

Pattern-based detectors identify gendered role stereotypes and gender-profession associations. When bias is found, the UI surfaces actionable recommendations (e.g., “avoid associating professions with specific genders”).

H. Bias patterns

We focus on common stereotypes that can appear in short narratives. Table V gives examples of patterns used to flag potential issues. The detector is intentionally conservative and aims to highlight questionable associations rather than make definitive claims about bias.

I. Explainability

We expose three complementary interpretation methods:

TABLE V
EXAMPLES OF BIAS PATTERNS (SIMPLIFIED).

Category	Example pattern
Gender roles	“women should...”, “men must...”
Profession stereotypes	“woman nurse”, “man engineer”
Trait stereotyping	“emotional woman”, “tough man”

Attention Visualization: Heatmaps over the last layer (or user-selected layer) showing which tokens the model attends to during generation.

Leave-One-Out Token Importance: Measures each token’s contribution by computing the KL divergence between baseline and masked predictions:

$$\begin{aligned} \text{importance}(t_i) &= D_{KL}(P_{\text{baseline}} \| P_{\text{masked}}) \\ &= \sum P_{\text{baseline}} \cdot \log \frac{P_{\text{baseline}}}{P_{\text{masked}}} \end{aligned}$$

Gradient-Based Importance: Alternative method using embedding gradients:

$$\text{importance}(t_i) = \|\nabla_{\text{embeddings}} \mathcal{L}\|_2$$

Additionally, we provide next-token probability tables with entropy-based uncertainty. All methods are available in the UI and can be exported programmatically.

J. Interpretation caveats

Attention is not a complete explanation of model behavior; it is best treated as a diagnostic visualization rather than a causal attribution. Leave-one-out importance is more faithful to the model output but is computationally expensive. We therefore expose both approaches and allow users to choose depending on context.

VIII. IMPLEMENTATION AND REPRODUCIBILITY

A. Offline mode

To support grading environments with limited connectivity, the implementation includes an offline mode that avoids run-time downloads. When enabled, the system sets Hugging Face to `local_files_only` and disables Detoxify by default. If a local fine-tuned model is available, the application uses it; otherwise the system instructs the user to cache the base model beforehand.

B. UI workflow

The Gradio interface is designed as a guided workflow for coursework:

- **Generate:** choose genre and sampling settings, then generate a story.
- **Inspect:** view the content rating, confidence, and any warnings (safety or bias).
- **Explain:** inspect the next-token distribution, attention heatmap, and token-importance plot.
- **Revise:** adjust prompt or parameters and regenerate to observe changes.

This interaction pattern encourages safe experimentation and makes model behavior more transparent for students.

TABLE VI
EVALUATION ON STORYTELLING SET (50 SAMPLES). LOWER PPL IS BETTER; HIGHER SAFE IS BETTER.

Model	PPL ↓	Safe ↑	Bias ↓
GPT-2 base	41.12	1.00	0.00
GPT-2 (ft. WritingPrompts)	35.30	1.00	0.00

C. Testing strategy

Unit tests isolate modules via mocking to avoid loading large models. Slow or environment-dependent benchmarks are marked separately and can be skipped during routine runs. A smoke test ensures that the bundled dataset loads and that offline initialization does not trigger model downloads.

D. Computational complexity

Regex filtering runs in time roughly linear in the text length for each pattern set. Bias detection similarly scales with pattern count. The dominant cost is language model inference, which scales with sequence length and model size. Explainability adds overhead: attention extraction is one forward pass with `output_attentions`, while leave-one-out requires multiple passes (one per token), making it most suitable for short prompts or classroom demonstrations.

IX. EVALUATION

A. Metrics

We report three primary metrics:

Perplexity (PPL): Measures model confidence in generated text:

$$\text{PPL} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(\text{token}_i | \text{context}) \right)$$

Safe Content Ratio: Proportion of outputs passing safety filters:

$$\text{Safe Ratio} = \frac{\text{safe_count}}{\text{total_count}}$$

Bias Ratio: Proportion of outputs containing detected stereotypes:

$$\text{Bias Ratio} = \frac{\text{biased_count}}{\text{total_count}}$$

Efficiency is measured as tokens/s on CPU and GPU.

B. Experimental setup

We evaluate on the bundled local dataset to ensure offline reproducibility. Unless otherwise stated, we generate 200 tokens with temperature 0.8, top-*k* 50, and top-*p* 0.95. Safety is computed using the same filter stack used in the application. Bias incidence is computed using pattern matches for gender-role and profession stereotypes.

TABLE VII
MEASURED GENERATION THROUGHPUT (TOKENS/S) ON APPLE SILICON
M-SERIES.

Model	CPU	MPS (GPU)
GPT-2	79	62
DistilGPT-2	115	76

TABLE VIII
ABLATION ON ETHICAL COMPONENTS (27 TEST SAMPLES INCLUDING
EDGE CASES).

Setting	Unsafe Caught	Bias Caught	Total Issues
No filter	0	0	0 (no detection)
Regex-only	1	0	1
+ Bias checks	1	3	4
+ Detoxify	4	3	7

C. Results

Table VI compares base GPT-2 with a WritingPrompts fine-tuned model on the local evaluation set (50 samples).

On the full 50-sample local dataset, our evaluation script reported perplexity 41.12 for base GPT-2 and 35.30 for the fine-tuned model (14% improvement), with safe ratio 100% and bias ratio 0% under regex-only safety checks. These values demonstrate reproducibility of the evaluation pipeline.

D. Throughput

Table VII summarizes throughput for typical hardware.

E. Ablation study

We evaluate the impact of each ethical layer on 27 test samples (19 curated edge cases plus 8 GPT-2 generated stories). Table VIII shows the cumulative effect: regex patterns catch explicit harmful content (e.g., violence keywords), bias detection identifies 3 stereotypical patterns, and Detoxify catches 3 additional toxic samples that passed regex checks. The full pipeline flags 4 unsafe and 3 biased samples, leaving 21/27 (78%) fully clean.

F. Qualitative example

Figure 2 shows a short illustrative output excerpt. The system surfaces a safety rating and suggests revisions when sensitive topics are detected, enabling a “generate–inspect–revise” workflow suitable for coursework.

G. Efficiency

On Apple Silicon (M-series), GPT-2 generation throughput reached ~79 tokens/s on CPU and ~62 tokens/s on MPS (GPU). DistilGPT-2 achieved ~115 tokens/s on CPU. Note that for small models, CPU can outperform MPS due to kernel overhead; NVIDIA GPUs would show higher GPU throughput. Detoxify runs once per generation and is cached to reduce latency in the UI.

Prompt: “In a magical kingdom, a young wizard...”
Output (excerpt): “... they chose kindness over revenge and helped the village rebuild ...”
Safety: SAFE (confidence 0.85) **Bias:** none detected

Fig. 2. Qualitative example with surfaced safety/bias diagnostics.

H. Error analysis

Most false positives arise from keyword-based sensitive-topic detection (e.g., fictional “attack” scenes in adventure stories) and can be mitigated by refining patterns or adding context-aware classifiers. False negatives may occur for implicit harm or subtle stereotypes that are not captured by patterns; addressing this requires semantic safety models and larger bias benchmarks.

X. CASE STUDY

We present three small, reproducible examples generated and analyzed using the project code.

A. Story generation and safety rating

Prompt: “In a magical kingdom,” with fantasy prompting enabled. The system generated the following excerpt:

In a magical kingdom far beyond the mountains, In a magical kingdom, there are no other souls. There is only one way of being in that world... A single soul! That’s how you can be born into this universe!
 ...

The ethical filter labeled this output as **Mild** (safe) with confidence 0.70, and suggested adding more explicitly positive story elements.

B. Bias detection example

For the sentence “The woman nurse helped the man engineer while the emotional woman waited.” the bias detector flagged both gender-role phrasing and profession stereotypes, returning recommendations to use more balanced representations and avoid associating professions with specific genders.

C. Unsafe content example (blocked)

For the sentence “He planned to kill him and everyone cheered.” the filter matched a blocked pattern and assigned the **Blocked** rating with high confidence (0.95). This illustrates how explicit violent instructions are detected with simple, transparent rules.

XI. DISCUSSION AND THREATS TO VALIDITY

A. Safety–creativity trade-off

Increasing safety strictness can reduce creative diversity, especially when regeneration is triggered repeatedly. In practice, we expose generation parameters (temperature/top- p) and encourage users to tune them with awareness of safety constraints.

B. Threats to validity

Results are based on a small local dataset and lightweight heuristics; conclusions may not generalize to broader domains or languages. Automatic safety metrics reflect the chosen detectors and thresholds. Human evaluation is limited in size and should be interpreted as indicative rather than definitive.

XII. ETHICAL CONSIDERATIONS

Safety: Multi-layer filtering blocks high-toxicity or violent content and suggests revisions when partial issues are found. Bias: Pattern-based checks highlight stereotypical associations; users are prompted to rephrase. Transparency: Explainability artifacts accompany outputs to foster user trust. Limitations include incomplete coverage of nuanced harms and English-centric patterns; human review remains recommended for sensitive deployments.

XIII. LIMITATIONS AND FUTURE WORK

Current bias detection is rule-based and English-focused; multilingual and semantic bias detection are future targets. Dataset size is modest; scaling to larger, diverse corpora should improve robustness. RLHF-style safety tuning and controllable generation could further reduce unsafe completions. UI performance can benefit from quantization and streaming for low-resource devices.

XIV. CONCLUSION

The Ethical AI Storyteller demonstrates that safety, bias awareness, and interpretability can be integrated into neural story generation without sacrificing fluency. Our open pipeline—from fine-tuning to evaluation and UI—supports reproducible coursework experiments and serves as a foundation for ethical, creative NLP applications.

ACKNOWLEDGMENT

We thank the NLP 2025 course staff at the University of Prishtina for guidance and feedback.

REFERENCES

- [1] A. Radford et al., “Language Models are Unsupervised Multitask Learners,” OpenAI, 2019.
- [2] T. Brown et al., “Language Models are Few-Shot Learners,” NeurIPS, 2020.
- [3] L. Hanu and Unitary Team, “Detoxify,” GitHub, 2020.
- [4] E. Bender et al., “On the Dangers of Stochastic Parrots,” FAccT, 2021.
- [5] E. Sheng et al., “Towards Controllable Biases in Language Generation,” EMNLP, 2020.
- [6] J. Vig, “A Multiscale Visualization of Attention in the Transformer Model,” ACL Demo, 2019.
- [7] L. Arras et al., “Explaining Recurrent Neural Network Predictions in Sentiment Analysis,” EACL Workshop, 2017.
- [8] A. Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017.
- [9] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” EMNLP: System Demonstrations, 2020.
- [10] W. McKinney, “Data Structures for Statistical Computing in Python,” Proc. SciPy, 2010.
- [11] S. Gehman et al., “RealToxicityPrompts: Evaluating Neural Toxic Generation in Language Models,” Findings of EMNLP, 2020.
- [12] N. Nangia et al., “CrowS-Pairs: A Challenge Dataset for Measuring Social Biases in Masked Language Models,” EMNLP, 2020.
- [13] T. Bolukbasi et al., “Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings,” NeurIPS, 2016.

- [14] J. Li et al., “What Does BERT Look At? An Analysis of BERT’s Attention,” ACL Workshop BlackboxNLP, 2019.